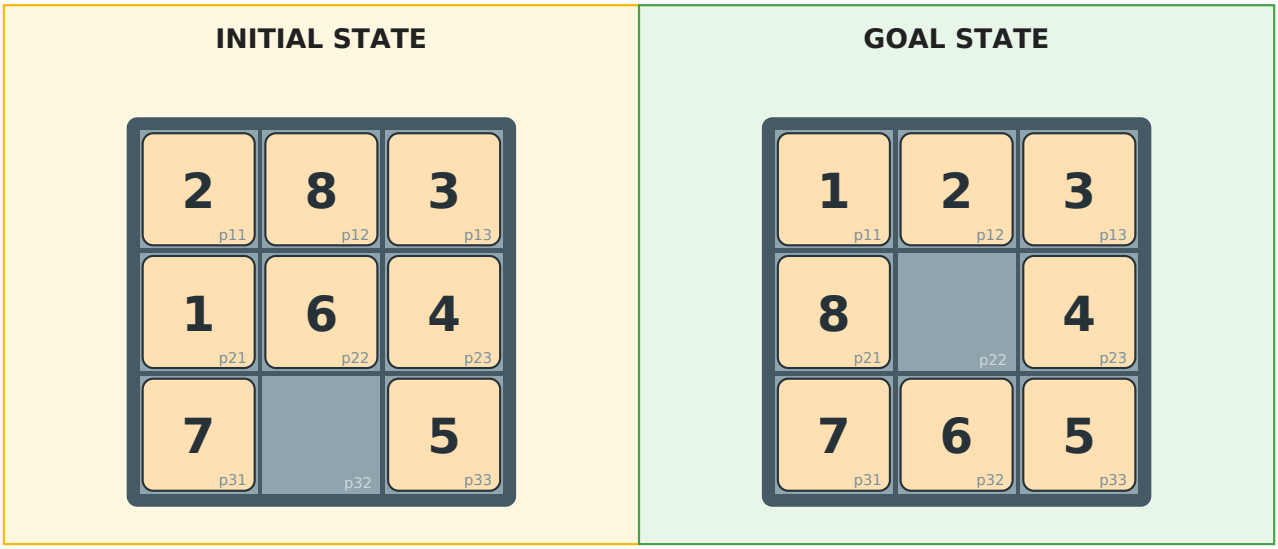
Problem 9: 8-Puzzle

Level: **Medium** · Folder: **09_Eight_Puzzle/** · Files: domain.pddl, problem.pddl, solution_plan.txt

1. The problem

Eight numbered tiles sit on a 3 x 3 board with one empty square. A tile next to the empty square can slide into it. Reach the goal arrangement from the start arrangement. This is the textbook instance from Nilsson's *Principles of Artificial Intelligence*.



The goal picture shows one state that satisfies the goal (the final state of the plan). Only the facts in the :goal section are required; anything else may be different.

What you will learn

- Modelling a **grid** with named positions and an *adjacent* relation.
- A large state space: 9!/2 = 181,440 reachable states.
- Why **heuristic search** (A*) matters compared with blind search.

2. Formulating the problem in PDDL

Step 1: Types and objects. tile: t1 to t8. position: p11 to p33 (p<row><column>).

Step 2: Predicates describe what can be true or false in a state:

Predicate	Meaning
`(at ?t ?p)`	Tile ?t is on square ?p.
`(empty ?p)`	Square ?p is the blank.
`(adjacent ?p1 ?p2)`	Squares ?p1 and ?p2 share an edge (static, 24 facts).

Step 3: Actions. Each action has *parameters*, a *precondition* (what must be true to apply it) and an *effect* (facts that become **true (+)** or **false (−)**):

Action	Precondition	Effect

slide
`?t - tile, ?from - position, ?to - position`
Slide tile ?t from ?from into the neighbouring empty square ?to.
	`(at ?t ?from)`
	`(empty ?to)`
	`(adjacent ?from ?to)`
		`+ (at ?t ?to)`
		`+ (empty ?from)`
		`− (at ?t ?from)`
		`− (empty ?to)`

Step 4: Initial state (`:init`) lists *every* fact that is true at the start. Everything else is false (closed-world assumption). **Step 5: Goal** (`:goal`) lists the facts that must be true at the end.

INITIAL STATE (33 facts)		GOAL

`(at t2 p11)`
`(at t8 p12)`
`(at t3 p13)`
`(at t1 p21)`
`(at t6 p22)`
`(at t4 p23)`
`(at t7 p31)`
`(empty p32)`
`(at t5 p33)`
`(adjacent p11 p12)`
`(adjacent p12 p11)`
`(adjacent p12 p13)`
`(adjacent p13 p12)`
`(adjacent p21 p22)`
`(adjacent p22 p21)`
`(adjacent p22 p23)`
`(adjacent p23 p22)`
	`(adjacent p31 p32)`
	`(adjacent p32 p31)`
	`(adjacent p32 p33)`
	`(adjacent p33 p32)`
	`(adjacent p11 p21)`
	`(adjacent p21 p11)`
	`(adjacent p21 p31)`
	`(adjacent p31 p21)`
	`(adjacent p12 p22)`
	`(adjacent p22 p12)`
	`(adjacent p22 p32)`
	`(adjacent p32 p22)`
	`(adjacent p13 p23)`
	`(adjacent p23 p13)`
	`(adjacent p23 p33)`
	`(adjacent p33 p23)`
		`(at t1 p11)`
		`(at t2 p12)`
		`(at t3 p13)`
		`(at t8 p21)`
		`(at t4 p23)`
		`(at t7 p31)`
		`(at t6 p32)`
		`(at t5 p33)`

Modelling notes

- People often describe the puzzle as 'moving the blank'. In PDDL it is easier to move a **tile**, because the tile is an object. Sliding a tile left is the same as moving the blank right.
- The board edges are modelled simply by leaving out adjacency facts. There is no explicit wall.
- Not every arrangement can be solved. Half of all permutations are unreachable (a parity argument). A planner will search the whole reachable space before it reports 'no solution'.

3. Solution plan (5 actions, optimal)

5 slides solve this instance optimally. The pictures below show the state **after** each action. They were produced by executing the plan on the PDDL model.



Initial state



Step 1
`(slide t6 p22 p32)`



Step 2
`(slide t8 p12 p22)`



Step 3
`(slide t2 p11 p12)`



Step 4
`(slide t1 p21 p11)`



Step 5
`(slide t8 p22 p21)`

4. How to run it

Online (no installation)	Open **editor.planning.domains**, paste *domain.pddl* and *problem.pddl* into two tabs and press **Solve**.
pyperplan (Python)	`pip install pyperplan` `pyperplan domain.pddl problem.pddl` (breadth-first search) `pyperplan -s astar -H lmcut domain.pddl problem.pddl` (optimal A*)
Fast Downward	`fast-downward.py domain.pddl problem.pddl --search "astar(lmcut())"`

5. Lab exercises

1. Make a harder start state by applying 15-20 random slides to the goal. Compare `pyperplan -s bfs` with `-s astar -H lmcut`.
2. Swap tiles 1 and 2 in the start state and run the planner. Explain the result.
3. Extend the model to the 15-puzzle (4 x 4). How many adjacency facts do you need?

6. PDDL source code (appendix)

domain.pddl

```
;; ============================================================
;; Problem 9 : 8-Puzzle (sliding tiles)
;; Level     : Medium
;; Eight numbered tiles on a 3x3 board with one empty square.
;; A tile can slide into the empty square next to it.
;; ============================================================
(define (domain eight-puzzle)
  (:requirements :strips :typing)

  (:types tile position)

  (:predicates
    (at ?t - tile ?p - position)              ; tile ?t is on square ?p
    (empty ?p - position)                     ; square ?p is the blank
    (adjacent ?p1 - position ?p2 - position)   ; squares share an edge
  )

  ;; Slide tile ?t from ?from into the empty neighbouring square ?to
  (:action slide
    :parameters (?t - tile ?from - position ?to - position)
    :precondition (and (at ?t ?from)
                       (empty ?to)
                       (adjacent ?from ?to))
    :effect (and (at ?t ?to)
                 (empty ?from)
                 (not (at ?t ?from))
                 (not (empty ?to)))
  )
)
```

problem.pddl

```
;; Positions are named p<row><col>:
;;    p11 p12 p13
;;    p21 p22 p23
;;    p31 p32 p33
;;
;;   Initial        Goal
;;   2 8 3          1 2 3
;;   1 6 4          8 _ 4
;;   7 _ 5          7 6 5
(define (problem eight-puzzle-1)
  (:domain eight-puzzle)

  (:objects
    t1 t2 t3 t4 t5 t6 t7 t8          - tile
    p11 p12 p13 p21 p22 p23 p31 p32 p33 - position
```

```
  )

  (:init
    ;; board layout
    (at t2 p11) (at t8 p12) (at t3 p13)
    (at t1 p21) (at t6 p22) (at t4 p23)
    (at t7 p31) (empty p32) (at t5 p33)

    ;; horizontal neighbours
    (adjacent p11 p12) (adjacent p12 p11) (adjacent p12 p13) (adjacent p13 p12)
    (adjacent p21 p22) (adjacent p22 p21) (adjacent p22 p23) (adjacent p23 p22)
    (adjacent p31 p32) (adjacent p32 p31) (adjacent p32 p33) (adjacent p33 p32)
    ;; vertical neighbours
    (adjacent p11 p21) (adjacent p21 p11) (adjacent p21 p31) (adjacent p31 p21)
    (adjacent p12 p22) (adjacent p22 p12) (adjacent p22 p32) (adjacent p32 p22)
    (adjacent p13 p23) (adjacent p23 p13) (adjacent p23 p33) (adjacent p33 p23)
  )

  (:goal (and
    (at t1 p11) (at t2 p12) (at t3 p13)
    (at t8 p21)             (at t4 p23)
    (at t7 p31) (at t6 p32) (at t5 p33)))
)
```

solution_plan.txt

1. (slide t6 p22 p32)
2. (slide t8 p12 p22)
3. (slide t2 p11 p12)
4. (slide t1 p21 p11)
5. (slide t8 p22 p21)